

Sistema de Tipos

Diseño e Implementación de un Lenguaje

v0.1.0

1. Chequeo de Tipos **1**
1.1. Coerción 4

1. Chequeo de Tipos

Si bien el uso de una tabla de símbolos permite recopilar los tipos de datos, y cualquier otra información relevante acerca de las variables y construcciones declaradas por el programador, la tabla por sí sola no dice nada acerca de la interrelación de tipos en una expresión compleja.

Por ejemplo, para la expresión `x + y`, la tabla tendrá un registro para la variable `x` y otro para la variable `y`, junto con su tipo de dato, pero... ¿ambos tipos son compatibles de tal modo que puedan utilizarse en una operación de suma mediante el símbolo +?

Para determinar la validez de un programa desde el punto de vista de su *sistema de tipos*, es necesario ejecutar un proceso de validación. A este proceso se lo denomina **type-checking**, y como su nombre lo indica, permite evaluar la validez de las construcciones desde el punto de vista de las operaciones y los tipos de datos involucrados en las mismas.

¿Cómo aplicar este proceso? Primero, es necesario comprender de qué manera un programa será válido desde el punto de vista de sus tipos de datos. Es posible considerar que existe una función `type(x)` que computa el tipo de dato para la construcción `x`. Aprovechando el hecho de que en esta etapa de la compilación ya se dispone de un AST válido desde el punto de vista de la sintaxis, una **construcción** se puede representar por un **nodo** en este árbol. Luego, `type(x)` podrá retornar el tipo del nodo `x`, o un tipo reservado que indique que la expresión está malformada (es decir, que tiene un tipo inválido), por ejemplo, el símbolo $\bot$ (a.k.a. *bottom*).

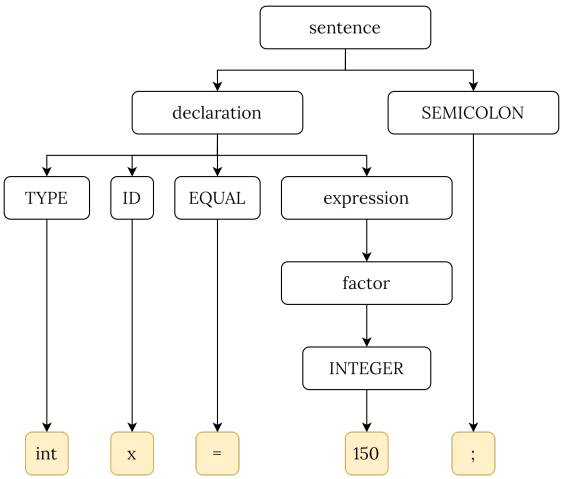
Concretamente, y suponiendo el siguiente fragmento de código en lenguaje Java:

```
00 int x = 150;
01 float y = 2.71f;
02 String z = “pepa”;
03
04 int u = x + 56;
05 float v = Math.ceil(u / y);
06 double w = v * z; // Type error.
```

Para la línea `01`, ¿cuál es el resultado de evaluar `type(int x = 150)`? Suponiendo que parte de la gramática es:

$$\textit{sentence} \longrightarrow \textit{declaration}\ \text{SEMICOLON}$$
$$\textit{declaration} \longrightarrow \text{TYPE ID EQUAL}\ \textit{expression}$$
$$\textit{expression} \longrightarrow \textit{factor}$$
$$\textit{factor} \longrightarrow \text{INTEGER}$$

entonces, el AST parcial para la construcción buscada debe ser:



Ahora bien, es posible desglosar el cómputo de la función `type(.)` de la siguiente manera:

```
type(sentence) = type(declaration)
type(declaration) = type(TYPE)
```

Lo cual es correcto, ya que el tipo de una *sentencia* en este caso es el tipo de la *declaración*, y el tipo de la declaración es el tipo de dato utilizado para, efectivamente, declarar una variable (`x` en este caso). El tipo que emita `type(TYPE)` será **`int`**, que bien puede ser un símbolo que identifique el tipo de dato de los números enteros (en términos programáticos, bien podría ser un símbolo de un `enum` en C).

El problema es que para la construcción utilizada, el proceso de obtención del tipo de dato no involucró la evaluación del nodo correspondiente a la expresión, con lo cual, si la expresión fuera inválida, aún así se retornaría el tipo **`int`** incorrectamente, cuando en realidad, se debería haber retornado $\perp$.

Esto significa que el cómputo de `type(x)` requiere del uso de una lógica adicional que evalúe la correctitud de las construcciones. Esta lógica deberá emitir ⊥ si `x` posee un tipo inválido, o el tipo correcto, en caso de que las construcciones sean válidas. En particular, y utilizando pseudocódigo para mostrar el programa de chequeo de tipos completo para esta parte del AST, se tiene:

```
type(sentence s) {
    return type(s.declaration)
}

type(declaration d) {
    if (type(d.TYPE) == type(d.expression)) {
        return type(d.TYPE)
    }
    else {
        log.error("The expression does not match the declaration type.")
        return ⊥
    }
}

type(TYPE t) {
    if (t.lexeme == "int") return int
    else if (t.lexeme == "float") return float
    else if (t.lexeme == "String") return string
    else {
        log.error("Unknown type: ${t.lexeme}.")
        return ⊥
    }
}

type(expression e) {
    return type(e.factor)
}

type(factor f) {
    return type(f.INTEGER)
}

type(INTEGER i) {
    return int
}
```

Con este pequeño *script*, ahora es posible obtener de manera recursiva el tipo de dato de cualquier sentencia, en particular de la buscada, pero además, verificar la validez de las mismas mediante la evaluación de `type(d.expression)` dentro de `type(declaration d)`. Si los tipos son equivalentes, entonces se emite el tipo correcto.

Es importante notar que la evaluación del tipo `type(INTEGER i)` siempre emite incondicionalmente el tipo **`int`**, lo cual es correcto, ya que el analizador léxico efectivamente generó dicho *token* porque identificó en la entrada un *lexema* de tipo entero, y por ende, es

válido retornar para dichos literales el tipo de dato directamente. Si no existieran definiciones de tipos que emiten el mismo directamente, entonces el script anterior nunca finalizaría su ejecución, y por lo tanto no sería un algoritmo, ya que ingresaría en un ciclo de llamados recursivos. Es fácil ver que estos casos representan los casos base de la recursión, y que siempre se asocian a nodos hoja en el AST.

Por otro lado, para la definición de `type(TYPE t)` se debe extraer el tipo definido dentro del lexema que este token acarrea (en este caso, representado por `t.lexeme`), ya que como se utilizó un único token para representar a todos los tipos de dato, no se puede identificar naturalmente de cuál de todos ellos se trata observando solo las reglas de producción. Obviamente es posible que no sea necesario emplear el lexema si es que el analizador léxico (*e.g.*, Flex) ya computó el tipo de dato y lo asignó dentro del token de otra manera más conveniente, en lugar de utilizar un *string* (por ejemplo, utilizando un `enum`).

En cuanto a la implementación de este sistema de tipos, se debe tener en cuenta que algunos lenguajes como C no poseen **method overloading**, y por lo tanto ya no es posible utilizar una misma función `type(.)`, sino que se deberá definir una para cada posible nodo en el árbol, es decir, en lugar de `type(sentence)`, debería ser `sentenceType(sentence)`, y así sucesivamente para cada tipo de nodo.

1.1. Coerción

El algoritmo de la sección anterior abre todo un abanico de posibilidades. Por ejemplo, la línea `05` del código original en Java, dice:

```
float v = Math.ceil(u / y);
```

En este caso, `u` es de tipo `int`, e `y` es de tipo `float`. Usualmente, los lenguajes realizan una **coerción** de tipo cuando es posible resolver las diferencias sin perder precisión en el cómputo. Sería válido que en esta situación, la variable `u` sea coaccionada a un tipo `float` para computar la división. Esto implica que el tipo generado será `float` en lugar de `int`, o incluso $\perp$ (si directamente no se soporta la coerción y los tipos deben ser exactamente iguales).

Suponiendo ahora las producciones:

$$\begin{gathered}\dots \\ \textit{expression} \longrightarrow \textit{factor} \\ \textit{expression} \longrightarrow \textit{factor DIV factor} \\ \textit{factor} \longrightarrow \textit{INTEGER} \\ \textit{factor} \longrightarrow \textit{FLOAT}\end{gathered}$$

Un mecanismo de *type-checking* **con coerción** sobre una expresión de división, se implementaría de esta manera:

```
type(expression e) {
    if (e.expressionType == FACTOR_EXPRESSION) {
        return type(e.factor)
    }
    else if (e.expressionType == DIV_EXPRESSION) {
```

```
            leftType = type(e.leftFactor)
            rightType = type(e.rightFactor)
            if (leftType = rightType) return leftType
            else {
                if ((leftType == int AND rightType == float)
                        OR (leftType == float AND rightType == int)) {
                    return float
                }
                else {
                    log.error("Incompatible types in expression.")
                    return ⊥
                }
            }
        }
        else {
            log.error("Abstract syntax tree malformed.")
            return ⊥
        }
    }
```

Nótese que en este script, cuando los tipos son equivalentes la coerción no tiene efecto, sino más bien cuando ambos difieren. En este caso, se verifican todas las combinaciones posibles, lo cual puede ser explosivo cuando se presentan múltiples tipos coaccionables o expresiones de mayor *aridad*. En estos casos, es preferible ordenar la dirección en la que se realizará la coerción, para reducir la lógica necesaria.

Por ejemplo, para la siguiente cadena de coacciones:

```
short int < int < long int < float < double
```

se asigna un índice numérico a cada uno de los tipos, es decir, short int = 0, int = 1, ..., double = 4. De este modo, se sabe que un tipo con índice n será coaccionable automáticamente a un tipo con índice m, solo si $n < m$.

Usando estas cadenas, la lógica se simplifica enormemente, ya que para una lista de tipos $T(1)$, ..., $T(i)$, y si los tipos son compatibles entre sí (pertenecen a la misma cadena de coerción), entonces $0 \leq \text{index}(T(k)) \leq 4$, y el tipo al cual se debe coaccionar debe ser el de mayor precisión dentro de dicha lista, tal que permita manipular los valores de todos los tipos involucrados, es decir, el tipo final debería ser $max(T(1), ..., T(i))$ (el que posee el máximo índice). Nótese que de esta manera, no es necesario comprobar todas las permutaciones posibles para determinar el tipo final, sino que el cómputo es proporcional a $O(i)$, siendo i el largo de la lista (es lineal, en lugar de cuadrático).